학습 노트: 매핑(Mapping)이란?

> 작성일: 2026-06-05 컨텍스트: concept_mapper.py — Bandit 룰 ID를 보안 개념으로 변환하는 코드 작성 중

Layer 1. 비유 — 중학생도 이해하는 설명

도서관에서 책을 찾을 때 "청구번호"를 씁니다.

- 청구번호 `813.6-김` → "한국 소설, 저자 김OO"
- 번호 자체는 의미 없지만, 표를 보면 즉시 어떤 책인지 알 수 있습니다.

매핑은 바로 그 "표**(Table)**" 입니다.

> "B608"이라는 암호 같은 번호 → "SQL Injection / raw_query"라는 사람이 이해하는 말로 바꿔주는 번역표.

Layer 2. 개념 — 공식 정의

매핑**(Mapping)** 이란, 한 집합의 원소를 다른 집합의 원소에 **1:1** 또는 **N:1**로 대응시키는 관계입니다.

수학적으로는 함수(Function) `f: A → B` 와 같습니다.

```
입력(Key)    →    출력(Value)
──────────────────────────────
"B608"       →    { keyword: "SQL Injection", sub_category: "raw_query",
"B324"       →    { keyword: "AUTH_VULNERABILITY", ... }
```

프로그래밍에서 매핑은 주로:

- 딕셔너리/해시맵 자료구조로 구현
- **Key**로 **Value**를 **O(1)**에 조회하는 것이 핵심

Layer 3. 왜 쓰나 — 없으면 생기는 문제

매핑이 없다면?

Bandit이 `B608`을 탐지했을 때, 코드 어딘가에서 이런 일이 벌어집니다:

```
# 매핑 없이 처리한다면 — 나쁜 예
if rule_id == "B608":
    keyword = "SQL Injection"
    sub_category = "raw_query"
    concept_id = "sql_injection"
elif rule_id == "B324":
    keyword = "AUTH_VULNERABILITY"
```

```
    sub_category = "weak_password_policy"
    concept_id = "..."
# ... Bandit 룰이 수백 개라면? → elif가 수백 줄
```

문제점:

1. 유지보수 지옥 — 룰 하나 추가할 때마다 elif 추가
2. 중복 — 같은 패턴이 여러 파일에 흩어짐
3. 가독성 저하 — 비즈니스 로직(매핑 데이터)이 코드 흐름에 묻힘
4. 테스트 어려움 — 데이터와 로직이 섞여 있어 단위 테스트 불가

Layer 4. 실제 코드 — concept_mapper.py 연결

```
# concept_mapper.py
RULE_CONCEPT_MAP = {
    "B608": {
        "keyword": "SQL Injection",
        "sub_category": "raw_query",
        "concept_id": "sql_injection",
        "mistake_type_concept_id": "sql_injection.raw_query",
        "confidence_score": 0.92,
    },
    "B324": {
        "keyword": "AUTH_VULNERABILITY",
        "sub_category": "weak_password_policy",
        ...
    }
}
```

이 코드에서 매핑이 하는 일

```
Bandit 탐지 결과                RULE_CONCEPT_MAP                파이프라인 출력
──────────────────  →  ──────────────────────────  →  ──────────────────
rule_id = "B608"        키로 딕셔너리 조회              keyword = "SQL Inject
                        (O(1) 시간복잡도)                concept_id = "sql_ir
                                                           confidence_score =
```

실제 조회 방법

```
def map_rule_to_concept(rule_id: str) -> dict | None:
    return RULE_CONCEPT_MAP.get(rule_id)  # 없으면 None 반환 (KeyError 없

# 사용
concept = map_rule_to_concept("B608")
# → { "keyword": "SQL Injection", "sub_category": "raw_query", ... }
```

왜 이 구조가 좋은가

항목	설명
데이터/로직 분리	RULE_CONCEPT_MAP은 순수 데이터, 조회 함수는 순수 로직
확장성	새 Bandit 룰 추가 = 딕셔너리에 항목 하나 추가